

Guía Pedagógica: Día 5 - Listados Dinámicos y el Nuevo Control Flow

Hoy daremos vida a nuestra página de inicio añadiendo múltiples categorías de películas. Hasta ahora, solo mostramos una película destacada en el "Hero". El objetivo de hoy es aprender a manejar colecciones de datos (arrays) y mostrarlas de forma elegante usando el nuevo **Control Flow** de Angular 17+ y **Signals**.

Contexto Pedagógico: Evolución en Angular

Durante mucho tiempo, en Angular utilizamos directivas estructurales como ***ngFor*** y ***ngIf*** para renderizar listas o mostrar/ocultar elementos. Angular 17 introdujo un nuevo flujo de control (***@for***, ***@if***, ***@switch***) que está integrado directamente en el framework (ya no necesitamos importar CommonModule en muchos casos), es mucho más fácil de leer y, lo más importante, ofrece un **rendimiento muy superior** al renderizar listas grandes.

Objetivo:

1. **Actualizar el Servicio:** Pedir a la API de TMDb las películas más populares.
2. **El Componente "Átomo" (MovieCard):** Una tarjeta reutilizable para una sola película.
3. **El Componente "Molécula" (MovieSlider):** Un contenedor con scroll horizontal para mostrar una lista de tarjetas.
4. **Integración:** Unir todo en nuestra página de Home.

Paso 1: Ampliando nuestro Servicio

Para mostrar más películas, necesitamos pedir las. Vamos a actualizar nuestro "mensajero", el MovieService.

¿Qué vamos a hacer?

Abre el archivo **src/app/core/services/movie.service.ts** y añade un nuevo método llamado **getPopularMovies()**.

```
import { HttpClient } from '@angular/common/http';
import { inject, Injectable } from '@angular/core';
import { environment } from '../../../environments/environment';
import { MovieResponse } from '../models/movie.model';

@Injectable({ providedIn: 'root' })
export class MovieService {
  private http = inject(HttpClient);
  private apiUrl = environment.baseUrl;

  getTrendingMovies() {
    return this.http.get<MovieResponse>
    (`${this.apiUrl}/trending/movie/day`);
  }

  // NUEVO MÉTODO
  getPopularMovies() {
    return this.http.get<MovieResponse>(`${this.apiUrl}/movie/popular`);
  }
}
```

Concepto Clave: Nota cómo reutilizamos **this.apiUrl** y el modelo **MovieResponse**. La estructura de respuesta de TMDB es consistente, lo que nos facilita muchísimo el trabajo.

Paso 2: Creando la Tarjeta de Película (MovieCard)

Vamos a crear el componente visual para una película individual.

1. Generar el componente

En tu terminal, ejecuta el comando para crear el componente dentro de la carpeta shared:

```
ng g c shared/components/movie-card
```

2. La Lógica (TypeScript)

Abre **movie-card.ts**. Este será un **Componente Presentacional (Dumb Component)**. Su única responsabilidad es recibir los datos de una película a través de un decorador `@Input` y mostrarlos. No pide datos a ninguna API por sí mismo.

```

import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Movie } from '../../../core/models/movie.model';

@Component({
  selector: 'app-movie-card',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './movie-card.html',
  styleUrls: ['./movie-card.css']
})
export class MovieCard {
  // Obligamos a que el componente padre SIEMPRE pase una película
  @Input({ required: true }) movie!: Movie;

  // Getter para construir la URL completa de la imagen de TMDB
  get posterUrl() {
    return this.movie.poster_path
      ? `https://image.tmdb.org/t/p/w500${this.movie.poster_path}`
      : 'assets/no-poster.png'; // Fallback por si no hay imagen
  }
}

```

3. La Vista (HTML)

Abre **movie-card.html**. Aquí usaremos el **Property Binding** ([propiedad]) para las imágenes y la interpolación (**{{ }}**) para los textos.

```

<div class="movie-card">
  <div class="card-image-container">
    <img
      [src]="posterUrl"
      [alt]="movie.title"
      class="card-image"
      loading="lazy"
    >
  </div>

  <div class="card-overlay">
    <h3 class="movie-title">{{ movie.title }}</h3>
    <div class="movie-rating">
      <span class="rating-icon">★</span>
      <!-- Usamos un Pipe para limitar los decimales de la puntuación -->
    >
    <span>{{ movie.vote_average | number:'1.1-1' }}</span>
  </div>
</div>

```

Tip de Performance: Nota el atributo **loading="lazy"** en la imagen. Es una excelente práctica de rendimiento web. Hace que el navegador solo descargue las imágenes cuando el usuario hace scroll y están a punto de entrar en la pantalla, ahorrando datos y acelerando la carga inicial.

4. Los Estilos (CSS)

Queremos que se vea como una aplicación premium (estilo Netflix/HBO). Copia este CSS en **movie-card.css**. Hemos añadido transiciones suaves y un overlay oscuro para que el texto siempre se pueda leer independientemente de los colores del póster.

```

.movie-card {
  position: relative;
  width: 200px;
  flex: 0 0 auto; /* Importante para que no se aplasten en el scroll horizontal */
  border-radius: 12px;
  overflow: hidden;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
  cursor: pointer;
  background-color: var(--card-bg, #1a1a1a);
}

.movie-card:hover {
  transform: scale(1.05); /* Efecto zoom al pasar el mouse */
  z-index: 10;
  box-shadow: 0 10px 20px rgba(0, 0, 0, 0.5);
}

.card-image-container {
  aspect-ratio: 2 / 3; /* Proporción clásica de un póster de cine */
  width: 100%;
}

.card-image {
  width: 100%;
  height: 100%;
  object-fit: cover;
  transition: opacity 0.3s ease;
}

.card-overlay {
  position: absolute;
  bottom: 0;
  left: 0;
  right: 0;
  padding: 1rem;
  background: linear-gradient(to top, rgba(0,0,0,0.9) 0%, transparent 100%);
  color: white;
  opacity: 0; /* Oculto por defecto */
  transition: opacity 0.3s ease;
}

/* Mostrar la información solo al hacer hover sobre la tarjeta */
.movie-card:hover .card-overlay {
  opacity: 1;
}

.movie-title {
  font-size: 0.9rem;
  margin-bottom: 0.25rem;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis; /* Pone "..." si el texto es muy largo para caber */
}

```

Paso 3: Creando el Carrusel (MovieSlider)

Ahora que tenemos nuestras tarjetas, las agruparemos en una fila desplazable.

1. Generar el componente

```
ng g c shared/components/movie-slider
```

2. La Lógica (TypeScript)

En **movie-slider.ts**, recibiremos una lista (array) de películas y un título opcional para la sección (ej. "Tendencias").

```
import { Component, Input } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Movie } from '../../../core/models/movie.model';
import { MovieCard } from '../movie-card/movie-card'; // Importamos el
hijo

@Component({
  selector: 'app-movie-slider',
  standalone: true,
  imports: [CommonModule, MovieCard], // ¡Registramos nuestra tarjeta
aquí!
  templateUrl: './movie-slider.html',
  styleUrls: ['./movie-slider.css']
})
export class MovieSlider {
  @Input({ required: true }) movies: Movie[] = [];
  @Input() title: string = '';
}
```

3. La Vista y el Nuevo @for (HTML)

En **movie-slider.html** veremos la magia de Angular 17. Despidete de *ngFor y *ngIf.

```
<section class="slider-container">
  @if (title) {
    <h2 class="slider-title">{{ title }}</h2>
  }

  <div class="slider-track">
    <!-- NUEVO CONTROL FLOW DE ANGULAR -->
    @for (movie of movies; track movie.id) {
      <app-movie-card [movie]="movie"></app-movie-card>
    } @empty {
      <p>Cargando películas o no hay resultados...</p>
    }
  </div>
</section>
```

¿Qué está pasando aquí?

- @for (item of lista; track identificador): Es la nueva forma de iterar. Angular EXIGE la instrucción track. Esto le dice al framework exactamente qué elemento cambió para que solo actualice ese elemento en el DOM, en lugar de destruir y recrear toda la lista entera. ¡Es brutal para el rendimiento!
- @empty: Un bloque salvavidas opcional. Si la lista llega vacía o está cargando, automáticamente muestra este bloque. Ya no tienes que lidiar con condiciones adicionales como *ngIf="movies.length === 0".

4. Estilos de Scroll Horizontal (CSS)

En **movie-slider.css**, el secreto está en usar flexbox (`display: flex`) y decirle al contenedor que permita desplazamiento horizontal (`overflow-x: auto`).

```
.slider-container {
  padding: 2rem 0;
  overflow: hidden;
}

.slider-title {
  font-size: 1.5rem;
  font-weight: 700;
  margin-bottom: 1.5rem;
  padding: 0 4%;
  color: white;
}

.slider-track {
  display: flex;
  gap: 1.25rem;
  padding: 0 4% 1.5rem 4%;
  overflow-x: auto;
  scroll-behavior: smooth;
  scrollbar-width: none; /* Oculta la barra de scroll nativa en Firefox */
}

/* Magia para ocultar la barra de scroll en navegadores basados en
Chrome/Safari */
.slider-track::-webkit-scrollbar {
  display: none;
}
```

Paso 4: Uniendo todo en Home (El Componente Padre)

Finalmente, le daremos vida a nuestra página principal consumiendo estos nuevos componentes.

1. Actualizando la Lógica de Home

Abre **src/app/features/home/home.ts**. Vamos a usar **Signals** para almacenar las listas de películas y pedir las al servicio tan pronto inicie el componente (`ngOnInit`).


```

import { Component, inject, OnInit, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { MovieService } from '../../core/services/movie.service';
import { Hero } from '../components/hero/hero';
import { MovieSlider } from '../../shared/components/movie-slider/movie-
slider';
import { Movie } from '../../core/models/movie.model';

@Component({
  selector: 'app-home',
  standalone: true,
  imports: [CommonModule, Hero, MovieSlider], // Agregamos MovieSlider a
  Las importaciones
  templateUrl: './home.html',
  styleUrls: ['./home.css'],
})
export class Home implements OnInit {
  private movieService = inject(MovieService);

  // Declaramos nuestras Signals para almacenar el estado de forma
  reactiva
  featuredMovie = signal<Movie | null>(null);
  trendingMovies = signal<Movie[]>([]);
  popularMovies = signal<Movie[]>([]);

  ngOnInit(): void {
    // 1. Pedimos Las tendencias
    this.movieService.getTrendingMovies().subscribe({
      next: (data) => {
        if (data.results.length > 0) {
          this.featuredMovie.set(data.results[0]); // Ponemos La #1 como
  Destacada
          this.trendingMovies.set(data.results); // Guardamos La lista
  completa para el Slider
        }
      }
    });

    // 2. Pedimos Las populares
    this.movieService.getPopularMovies().subscribe({
      next: (data) => {
        this.popularMovies.set(data.results); // Guardamos La lista de
  populares
      }
    });
  }
}

```

2. Actualizando el HTML de Home

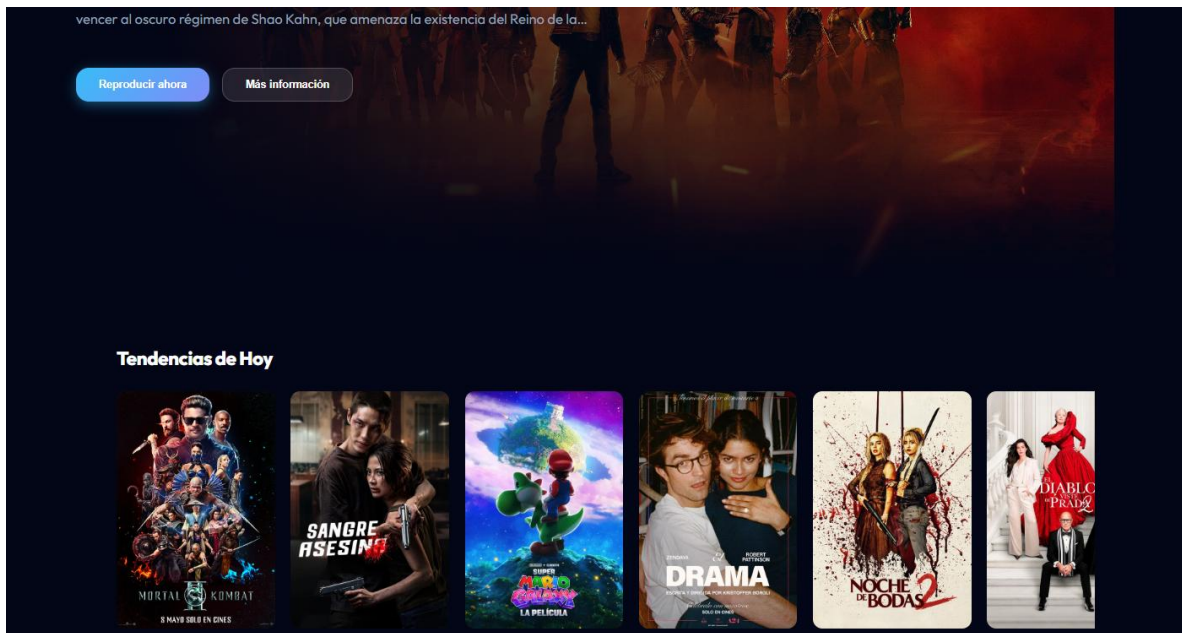
Abre **home.html** y añade nuestros nuevos sliders justo debajo del Hero. Usaremos la interpolación de las Signals (llamándolas con ()) para pasar los datos.

```
@if (featuredMovie(); as movie) {  
  <app-hero [movie]="movie"></app-hero>  
} @else {  
  <div class="loading-hero"></div>  
}  
  
<div class="home-sections">  
  <!-- Pasamos los datos de las Signals a los componentes hijos -->  
  <app-movie-slider  
    title="Tendencias de Hoy"  
    [movies]="trendingMovies()">  
  </app-movie-slider>  
  
  <app-movie-slider  
    title="Las más Populares"  
    [movies]="popularMovies()">  
  </app-movie-slider>  
</div>
```

Prueba de Aprendizaje (Checklist para el Aprendiz)

1. **¿Para qué sirve exactamente la cláusula track en un @for?** (Respuesta: Le da un identificador único a cada elemento en la iteración. Esto permite a Angular saber qué partes exactas del DOM debe actualizar si la lista cambia, sin tener que destruir y volver a dibujar toda la lista entera. Mejora drásticamente el rendimiento).
2. **¿Qué ventaja nos da usar loading="lazy" en la etiqueta ?** (Respuesta: Ahorra ancho de banda y tiempo de carga inicial, ya que el navegador solo descarga las imágenes cuando el usuario hace scroll hacia ellas y están a punto de hacerse visibles).
3. **¿Cómo logramos un scroll horizontal funcional y estético en CSS sin que la barra de scroll se vea fea?** (Respuesta: Usando overflow-x: auto en el

contenedor padre, flex: 0 0 auto en los hijos para que no colapsen, y ocultando la barra nativa usando selectores como `::-webkit-scrollbar { display: none; }`).



Control de Versiones

```
git add .  
git commit -m "feat: implementar movie cards y sliders usando control  
flow @for"  
git push origin main
```